

Fase 0 — Guía Completa del Intérprete de Pseudocódigo

Propósito: Explicar todos los archivos del intérprete (`.l` , `.y` , `ast.hpp` , `ast.cpp` , `init.hpp`) línea por línea, para que no tengas que ir preguntando durante el examen.

Escenario de examen: Normalmente el profesor os da `ast.hpp` y `ast.cpp` ya escritos. Vosotros solo editáis `interpreter.l` , `interpreter.y` y `table/init.hpp` . **Pero** en este Simulacro hemos preparado también la parte de AST (`.hpp` / `.cpp`) para que practiquéis todos los escenarios posibles. Si en el examen os pidieran modificar el AST, sabréis hacerlo.

Índice

1. [Arquitectura general](#)
2. [Fichero 1: `interpreter.l` \(Flex\)](#)
3. [Fichero 2: `interpreter.y` \(Bison\)](#)
4. [Fichero 3: `ast.hpp` \(AST — cabecera\)](#)
5. [Fichero 4: `ast.cpp` \(AST — implementación\)](#)
6. [Fichero 5: `table/init.hpp` \(tabla de símbolos\)](#)
7. [Flujo de evaluación](#)
8. [Trampas de examen](#)
9. [Checklist rápido](#)

1. Arquitectura general

El intérprete sigue el patrón clásico **Flex + Bison + AST**:

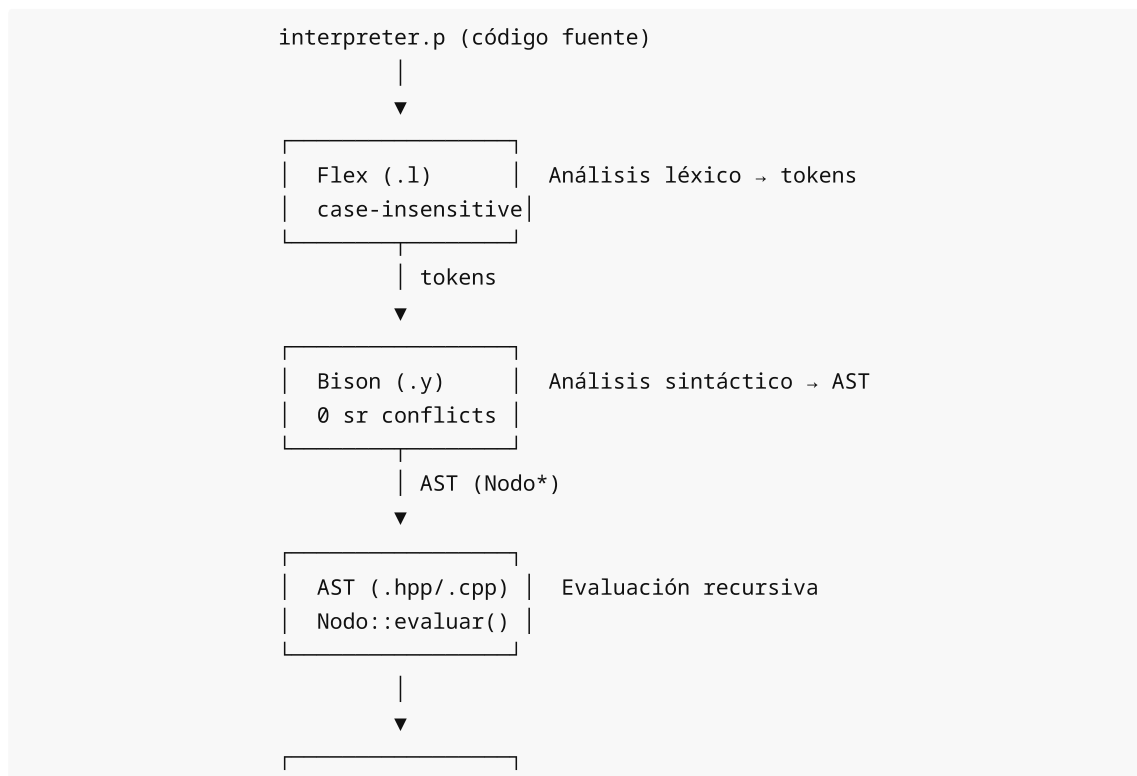


Tabla Símbolos (init.hpp)	Variables + tipos dinámicos
------------------------------	--------------------------------

Ficheros involucrados (los 5 que debes conocer):

#	Fichero	Rol	¿Se toca en examen?
1	parser/interpreter.l	Lexer (reconoce tokens)	Sí
2	parser/interpreter.y	Parser (construye AST)	Sí
3	ast/ast.hpp	Declaración de clases AST	Normalmente pre-hecho, pero debes saber modificarlo
4	ast/ast.cpp	Implementación de nodos AST	Normalmente pre-hecho, pero debes saber modificarlo
5	table/init.hpp	Registro de palabras reservadas	Sí

2. Fichero 1: `interpreter.l` (Flex)

Cabecera (`%{ ... %}`)

```
%{
#include <iostream>
#include <stdlib.h>
#include <string.h>

#include "../includes/macros.hpp"
#include "../ast/ast.hpp"
#include "interpreter.tab.h"
#include "../error/error.hpp"
#include "../table/table.hpp"
#include "../table/numericVariable.hpp"

extern lp::Table table;
extern int lineNumber;
%}
```

- **interpreter.tab.h** : generado por Bison. Define los tokens (`TOK_PRINT` , `TOK_IF` , etc.).
- **table** : tabla de símbolos global (declarada `extern` porque se define en `main.cpp`).
- **lineNumber** : contador de línea para mensajes de error.
- **macros.hpp** : contiene colores ANSI para `print` .

Opciones de Flex

```
%option case-insensitive
%option noyywrap
```

- **case-insensitive** : las palabras reservadas se reconocen en mayúsculas o minúsculas. **Obligatorio** para el lenguaje.
- **noyywrap** : no llamar a `yywrap()` al final del fichero (si el makefile no usa `-lfl`).
En el examen: el makefile del profesor usa `-lfl` , así que podéis **quitar** esta línea y confiar en `-lfl` . No pasa nada si la dejáis, `-lfl` es compatible con `noyywrap` .

Definiciones regulares

```

digito    [0-9]
letra     [a-zA-Z_]
id        {letra}({letra}|{digito})*
real      {digito}+\.{digito}+([E][+-]?{digito}+)?
cient     {digito}+\.{digito}*([E][+-]?{digito}+
entero    {digito}+
```

- **id** : identificador → empieza con letra/guion bajo, luego letras/dígitos/guiones bajos.
- **real** : número con decimales, opcionalmente con notación científica (`3.14e-5`).
- **cient** : notación científica sin decimales obligatorios (`1e10`).
- **entero** : dígitos sin punto.

Orden importa: Flex prueba las reglas en orden. Como `real` y `cient` aparecen antes que `entero` , un `3.14` se reconoce como `real` y no como `entero` .

Estados exclusivos (strings)

```
%x STRING
```

Estado para leer cadenas con comillas simples. Se activa con `'` y se desactiva con `'` de cierre.

Reglas de tokens (mapeo léxico)

```

";"      { return PUNTO_Y_COMA; }
"/ /"    { return DIVISION_ENTERA; }
"++"     { return INCREMENTO; }
"--"     { return DECREMENTO; }
"+:="    { return INCREMENTO_ASIGNACION; }
"-:="    { return DECREMENTO_ASIGNACION; }
```

Cada operador → un token. Esos tokens se definen en `interpreter.y` con `%token` .

Para palabras reservadas:

```

"read"      { return LEER; }
"read_string" { return LEER_CADENA; }
"print"     { return IMPRIMIR; }
...
```

Identificadores y números

```

{id}      {
            lp::Variable* var = table.lookupSymbol(yytext);
```

```

        if (var != NULL) {
            yylval.var = var;
            return IDENTIFICADOR;
        } else {
            // Error: variable no declarada
        }
    }
}

```

Clave: la tabla de símbolos distingue entre palabras reservadas y variables. Si `lookupSymbol` devuelve `NULL`, la variable no existe → error semántico.

Strings con escapes (manejados en el lexer)

```

{STRING_LIT}    {
    yytext[yytext-1] = '\0';          // quitar comilla final
    char* content = yytext + 1;      // saltar comilla inicial
    char* result = (char*)malloc(strlen(content) + 1);
    char* dest = result;
    char* src = content;

    while (*src) {
        if (*src == '\\') {
            src++;
            switch (*src) {
                case 'n': *dest = '\n'; break;
                case 't': *dest = '\t'; break;
                case '\\': *dest = '\\'; break;
                case '\': *dest = '\'; break;
                case '0': *dest = '\0'; break;
                default: *dest = *src; break;
            }
        } else {
            *dest = *src;
        }
        dest++; src++;
    }
    *dest = '\0';

    yylval.string = result;
    return STRING;
}

```

Las secuencias de escape se procesan AQUÍ, en el lexer, no en `ast.cpp`. El lexer recibe `'hola\nmundo'` con el regex `\"([^\"]|\\.)*\"` y lo transforma en `hola\nmundo` (con el newline real incrustado). El parser recibe el string ya resuelto y crea `StringNode` con él. `StringNode::evaluateString()` simplemente devuelve `this->_value` (inline en `ast.hpp`).

Comentarios

```

"(*)          { comment_level = 1; comentario(); }
<COMENTARIO>[^\n]*  { /* ignorar */ }

```

```
<COMENTARIO>"*)"      { if (--comment_level == 0) BEGIN INITIAL; }
<COMENTARIO>."        { /* ignorar */ }
```

Comentarios de bloque (* ... *) con contador de anidamiento (comment_level).

NO anidados según la especificación, pero igualmente se implementan con contador por si acaso.

Comentarios de línea:

```
"#".*      { /* ignorar hasta fin de línea */ }
```

Regla comodín (error léxico)

```
.      { Error::errorLexico("simbolo desconocido", yytext, lineNumber); }
```

Cualquier carácter no reconocido → error léxico.

3. Fichero 2: `interpreter.y` (Bison)

Cabecera C++ (%{ ... %})

```
%{
#include <iostream>
#include <string.h>
#include <math.h>

#include "../ast/ast.hpp"
#include "../error/error.hpp"
#include "../table/table.hpp"
#include "../table/numericVariable.hpp"

extern int yylex();
extern int lineNumber;
extern lp::Table table;

lp::Nodo* programa = NULL;

#define YYERROR_VERBOSE 1
%}
```

- **programa** : puntero global a la raíz del AST. Se usará en `main.cpp` para evaluar.
- **YYERROR_VERBOSE** : mensajes de error más descriptivos.

Declaraciones Bison

```
%union {
    char* str;
    lp::Nodo* nodo;
    lp::Variable* var;
    ...
}
```

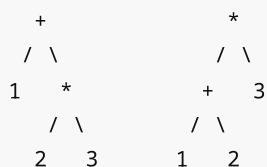
```
%token PUNTO_Y_COMA
%token DIVISION_ENTERA
...
%token <nodo> NUMERO CADENA BOOLEANO IDENTIFICADOR
%type <nodo> programa linea asignacion ...
```

- **%union** : tipos que pueden tener los símbolos. `nodo` para AST, `var` para variables, `str` para strings.
- **%token <tipo>** : los tokens tienen un tipo dentro de la unión.
- **%type <tipo>** : los no-terminales también tienen tipo.

Precedencia y asociatividad (explicación desde 0)

El problema

Bison es un parser LALR(1). Cuando ve `1 + 2 * 3`, hay DOS árboles sintácticos válidos:



Esto se llama **conflicto shift/reduce**. Bison no sabe si:

- **Reducir** `1 + 2` a `exp` (acción: `+`)
- **Shift** (leer) `*` (acción: `*`)

Para decidir, Bison mira la **precedencia** y **asociatividad** declaradas.

%left , %right , %nonassoc — declarar operadores

```
%left '+' '-'      ← misma línea = misma precedencia; asociatividad izquierda
%right POTENCIA    ← asociatividad derecha
%nonassoc '<' '>'    ← no asociativo (a<b<c = error de sintaxis)
```

Cada línea es un nivel de precedencia. Primera línea = menor precedencia, última línea = mayor.

Tabla completa de este proyecto:

Línea	Asoc.	Operadores	Precedencia
1	%right	'?' ':' (ternario)	1 (mínima)
2	%left	OR	2
3	%left	AND	3
4	%nonassoc	EQUAL NOT_EQUAL GREATER_OR_EQUAL ... (relacionales)	4
5	%left	NOT	5
6	%left	PLUS MINUS (suma/resta)	6

7	%left	CONCAT (	
8	%left	MULTIPLICATION DIVISION MODULO DIV_ENTERA	8
9	%right	INCREMENTO DECREMENTO (++/--)	9
10	%nonassoc	UNARY	10
11	%right	POWER (^)	11 (máxima)

Cómo decide Bison cada conflicto

Cuando hay conflicto entre **reducir una regla R** y **shiftear un token T**:

```
exp '+' exp . '*' exp
      ↑
conflicto: reducir '+' (R) o shiftear '*' (T)?
```

Bison obtiene dos números:

1. **Precedencia del token** '*' → nivel 8
2. **Precedencia de la regla** (la regla `exp: exp '+' exp` hereda la del **último terminal** de su lado derecho = '+' → nivel 6)

Comparación	Acción	Ejemplo
Token > Regla	Shift (el nuevo operador manda)	* (8) > + (6) → 1 + (2 * 3) ✓
Token < Regla	Reduce (la operación pendiente termina)	+ (6) < ^ (11) → (1 + 2) ^ 3 ✓
Token = Regla + %left	Reduce (asociativo izq.)	1 - 2 - 3 → (1 - 2) - 3 ✓
Token = Regla + %right	Shift (asociativo der.)	2 ^ 3 ^ 4 → 2 ^ (3 ^ 4) ✓
Token = Regla + %nonassoc	Error sintáctico	a < b < c → error

%prec — override manual de precedencia

A veces la precedencia que Bison deduce automáticamente no es la que queremos. `%prec` fuerza a una regla a usar una precedencia **diferente**:

```
exp: '-' exp %prec UNARY
```

Sin `%prec`: la regla `exp: '-' exp` heredaría la precedencia del último terminal de su lado derecho, que es `exp` (un no-terminal, no tiene precedencia → 0).

Con `%prec UNARY`: la regla usa la precedencia del token `UNARY` (nivel 10, muy alta).

¿Por qué hace falta? Para que `-3 + 5` se parse como `(-3) + 5`, no como `-(3 + 5)`. El menos unario debe tener **más** precedencia que el menos binario.

El caso concreto del ternario ?:

```
1 < 2 ? 10 : 20
```

El parser llega a: `exp '<' exp . '?' exp ':' exp`. El `.` entre `2` y `?` es el conflicto:

- **Reducir:** `1 < 2` a `exp` (usar el relacional `<`)
- **Shift:** leer `?` (empezar el ternario)

Bison compara:

- **Token** `'?'` → nivel 1 (el más bajo de todos)
- **Regla** `exp: exp '<' exp` → hereda de `'<'` → nivel 4

Token (1) < Regla (4) → Reduce → `(1 < 2) ? 10 : 20` ✓

Si `%right '?' ':'` estuviera entre los relacionales y los aritméticos:

```
%left '<' '>'          ← nivel 1
%right '?' ':'          ← nivel 2 ← MAL, mayor que '<'
%left '+' '-'          ← nivel 3
```

`'?'` (nivel 2) > `'<'` (nivel 1) → **Shift** → `1 < (2 ? 10 : 20)` ✗

Regla de oro: `%right '?' ':'` siempre el primero, con la menor precedencia.

El `%prec` en el ternario

```
exp: exp '?' exp ':' exp %prec '?'
```

El `%prec '?'` está aquí por seguridad. El último terminal de la regla es `':'`, y si `':'` tuviera una precedencia diferente a `'?'`, Bison usaría la de `':'`. Con `%prec '?'` forzamos a que use la de `'?'` explícitamente.

Regla base

```
programa: linea programa
        | linea
        ;

linea: sentencia PUNTO_Y_COMA
     | sentencia
     ;
```

`programa` es una secuencia de líneas. Cada línea es una sentencia seguida opcionalmente de `PUNTO_Y_COMA ( ; )`.

El `;` es **opcional** en la última línea, pero recomendable siempre.

Categorías de sentencias

```
sentencia: bloque_if
         | bloque_while
         | bloque_repeat
         | bloque_for
         | bloque_switch
```



```

| sentencia_asignacion
| sentencia_lectura
| sentencia_escritura
| sentencia_incdec
| ...
;

```

Cada una es una regla que devuelve un `Nodo*` :

```

sentencia_asignacion: IDENTIFICADOR ASIGNACION expresion
{ $$ = new lp::AssignStmt($1, $3); }
;

```

Asignaciones compuestas (`+=` , `-=`)

```

sentencia_incdec: IDENTIFICADOR INCREMENTO_ASIGNACION expresion
{ $$ = new lp::AssignIncrementStmt($1, $3); }
;

```

while

```

bloque_while: WHILE expresion DO sentencia END_WHILE
{ $$ = new lp::WhileStmt($2, $4); }
;

```

do ... while (sin end_while)

```

bloque_do_while: DO sentencia WHILE expresion
{ $$ = new lp::DoWhileStmt($4, $2); }
;

```

Sin end_while — la gramática del `do ... while` termina en `WHILE expresion` .

repeat ... until

```

bloque_repeat: REPEAT sentencia UNTIL expresion
{ $$ = new lp::RepeatStmt($2, $4); }
;

```

if con posible else

```

bloque_if: IF expresion THEN sentencia ELSE sentencia END_IF
{ $$ = new lp::IfStmt($2, $4, $6); }
| IF expresion THEN sentencia END_IF

```

```
{ $$ = new lp::IfStmt($2, $4); }  
;
```

El `%nonassoc THEN` (no mostrado) evita el conflicto shift/reduce del *dangling else*.

Aunque en este lenguaje tenemos `END_IF`, así que el conflicto no existe.

for

```
bloque_for: FOR IDENTIFICADOR FROM expresion TO expresion STEP expresion DO  
sentencia END_FOR  
{ $$ = new lp::ForStmt($2, $4, $6, $8, $10); }  
  | FOR IDENTIFICADOR FROM expresion TO expresion DO sentencia END_FOR  
{ $$ = new lp::ForStmt($2, $4, $6, $8); }  
;
```

Dos versiones: con `STEP` y sin `STEP` (por defecto paso 1).

switch

```
bloque_switch: SWITCH expresion casos_switch END_SWITCH  
{ $$ = new lp::SwitchStmt($2, $3); }  
;  
  
casos_switch: CASE expresion DOS_PUNTOS sentencia casos_switch  
{ /* añadir caso a la lista */ }  
  | DEFAULT DOS_PUNTOS sentencia  
{ /* caso por defecto */ }  
  | /* vacío */  
{ $$ = NULL; }  
;
```

Expresiones (árbol)

```
expresion: expresion OR expresion      { $$ = new lp::OrNode($1, $3); }  
  | expresion AND expresion           { $$ = new lp::AndNode($1, $3); }  
  | NOT expresion                     { $$ = new lp::NotNode($2); }  
  | expresion '=' expresion           { $$ = new lp::EqualNode($1, $3); }  
  | expresion "<>" expresion            { $$ = new lp::NotEqualNode($1, $3); }  
  | expresion '<' expresion             { $$ = new lp::LessNode($1, $3); }  
  | expresion '>' expresion             { $$ = new lp::GreaterNode($1, $3); }  
  | expresion "<=" expresion            { $$ = new lp::LessEqualNode($1, $3); }  
  | expresion ">=" expresion            { $$ = new lp::LessEqualNode($1, $3); }  
  | expresion '+' expresion           { $$ = new lp::AdditionNode($1, $3); }  
  | expresion '-' expresion           { $$ = new lp::SubtractionNode($1, $3); }  
  | expresion '*' expresion           { $$ = new lp::MultiplicationNode($1, $3); }  
  | expresion '/' expresion           { $$ = new lp::DivisionNode($1, $3); }  
  | expresion DIVISION_ENTERA expresion { $$ = new lp::IntegerDivisionNode($1,  
$3); }  
  | expresion MOD expresion           { $$ = new lp::ModulusNode($1, $3); }
```

```

        | expresion '^' expresion      { $$ = new lp::PotenciaNode($1, $3); }
        | '-' expresion %prec MENOS_UNARIO { $$ = new lp::UnaryMinusNode($2); }
        | expresion '?' expresion ':' expresion %prec '?' { $$ = new
lp::TernaryNode($1, $3, $5); }
        | '(' expresion ')'           { $$ = $2; }
        | NUMERO                      { $$ = $1; }
        | CADENA                      { $$ = $1; }
        | BOOLEANO                   { $$ = $1; }
        | IDENTIFICADOR               { $$ = $1; }
        | funcion_predefinida         { $$ = $1; }
        ;

```

Notas:

- `DIVISION_ENTERA` es el token para `//`.
- `%prec '?'` en el ternario fuerza la precedencia correcta.
- `'?'` y `':'` se declaran con `%right` (asociativo por la derecha) para permitir ternarios anidados.

++ y -- (pre-incremento/decremento)

```

sentencia_incdec: INCREMENTO IDENTIFICADOR
{ $$ = new lp::PreIncrementStmt($2); }
| DECREMENTO IDENTIFICADOR
{ $$ = new lp::PreDecrementStmt($2); }
;

```

Cómo garantizar 0 conflictos (shift/reduce = 0)

El profesor compila con `make 2>&1 | grep conflict`. Si sale un solo conflicto, la pregunta vale 0.

¿Qué es un conflicto shift/reduce?

Es una ambigüedad en la gramática. El parser no sabe si debe:

- **Shift** (leer el siguiente token y apilarlo)
- **Reduce** (aplicar una regla gramatical a lo que ya tiene en la pila)

Bison resuelve muchos automáticamente con la precedencia. Los que no puede resolver se muestran como:

```
State 42 contains 9 shift/reduce conflicts.
```

Causa #1: `while` vs `do ... while` (la más común: 9 conflictos)

Problema: la gramática tiene el `while` normal y el `do ... while`. Al terminar el cuerpo del `do`, el parser encuentra `WHILE` y **no sabe** si ese `WHILE` es el final del `do-while` o el inicio de un `while` nuevo.

Solución: añadir `controlSymbol` (regla épsilon) después del `DO` :

```

controlSymbol: /* empty */ { control++; }
;

/* While normal */
while: WHILE controlSymbol exp DO stmtlist END_WHILE

```

```

    { $$ = new lp::WhileStmt($3, new lp::BlockStmt($5));
      control--; }
;

/* Do-while con DOS controlSymbol */
do_while: DO controlSymbol stmtlist WHILE controlSymbol exp SEMICOLON
    { $$ = new lp::DoWhileStmt(new lp::BlockStmt($3), $6);
      control--; control--; }
;

```

¿Por qué funciona? El primer `controlSymbol` (en el `DO`) incrementa `control` a 1. El parser ve `WHILE` después del cuerpo y:

- Si hay un `controlSymbol` antes → la reducción del cuerpo no se produce aún
- Como `control=1`, el código de `stmtlist` que ejecuta sentencias en modo interactivo se salta

El segundo `controlSymbol` (antes de `exp` en `do_while`) incrementa `control` a 2, y al final se decrementa dos veces.

Atención: al añadir `controlSymbol`, la numeración `$N` cambia en todas las reglas donde aparece. En `do_while`, el `$3` (`stmtlist`) pasa a ser `$3` (porque el primer `controlSymbol` cuenta como `$2`), y `$5` (`exp`) pasa a ser `$6`.

Causa #2: Dangling else (if sin END_IF)

Problema clásico:

```

stmt: IF exp THEN stmt
    | IF exp THEN stmt ELSE stmt
;

```

Cuando el parser ve `IF cond THEN IF cond THEN stmt ELSE stmt`, no sabe si el `ELSE` pertenece al primer `IF` o al segundo.

Solución: usar `%nonassoc THEN` y `%nonassoc ELSE` para dar menor precedencia a `THEN`:

```

%nonassoc THEN
%nonassoc ELSE

stmt: IF exp THEN stmt %prec THEN
    | IF exp THEN stmt ELSE stmt
;

```

`%nonassoc THEN` tiene menor precedencia que `%nonassoc ELSE`. Cuando hay conflicto entre reducir con la regla sin `ELSE` y shiftar `ELSE`, el `ELSE` gana (se asocia al `IF` más cercano).

En nuestro proyecto: no hay este conflicto porque todos los bloques se cierran con `END_IF`, `END_WHILE`, etc. El `END_*` desambigua.

Causa #3: Ambigüedad en expresiones

Si declaras mal la precedencia o falta algún operador, aparecen conflictos SR entre operadores.

Solución: declarar TODOS los operadores con `%left` , `%right` o `%nonassoc` en el orden correcto (ver sección de precedencia).

Reglas:

- Cada operador binario necesita una declaración de precedencia
- No mezcles niveles (ej. poner `'*'` y `'+'` en la misma línea)
- Los operadores unarios con `%prec` y un token ficticio (`UNARY` , `MENOS_UNARIO`)

Causa #4: Reglas épsilon sin control

Las reglas épsilon (`/* empty */`) pueden generar conflictos si el parser no sabe si aplicar la regla vacía o no.

Solución: las reglas épsilon solo deben aparecer donde sean estrictamente necesarias. Si una regla épsilon causa conflicto, considera reescribir la gramática.

Causa #5: Expresiones y sentencias mezcladas

No pongas reglas como:

```
stmt: exp ';' /* MAL: una expresion sola como sentencia */
```

Esto confunde al parser porque `exp` es a su vez `VARIABLE` , `NUMERO` , etc. Mejor separa claramente:

```
stmt: asgn ';' | print ';' | if | while | ...
exp:  ... solo expresiones ...
```

Checklist rápido para 0 sr conflicts

1. **✓ Precedencia:** cada operador tiene `%left` / `%right` / `%nonassoc` en el orden correcto
2. **✓ controlSymbol:** en `do_while` (y en `if` , `while` , `for` , `repeat` para el modo interactivo)
3. **✓ %prec:** en operadores unarios (`-` , `+`) y en el ternario
4. **✓ Separación:** sentencias y expresiones en no-terminales distintos
5. **✓ Sin reglas épsilon** innecesarias (solo `controlSymbol` y `stmtlist` vacío)
6. **✓ Probar:** `make 2>&1 | grep "conflict"` → no debe mostrar nada

Comando mágico para el examen:

```
make 2>&1 | grep -E "conflict|error"
```

Si no sale nada, tienes 0 conflictos y compila. Es lo primero que hay que hacer tras implementar cualquier cambio.

4. Fichero 3: `ast.hpp` (AST — cabecera)

Namespace `lp`

Todo el AST está dentro de `namespace lp` . Las clases principales:

```
lp::Nodo          (abstracta, virtual)
├─ lp::Programa
└─ lp::AssignStmt (asignación :=)
```

```

└─ lp::AssignIncrementStmt (+= / -=)
└─ lp::PreIncrementStmt (++)
└─ lp::PreDecrementStmt (--)
└─ lp::ReadStmt          (read / read_string)
└─ lp::PrintStmt         (print)
└─ lp::ClearStmt         (clear_screen)
└─ lp::PlaceStmt         (place)
└─ lp::IfStmt            (if-then-else)
└─ lp::WhileStmt         (while-do)
└─ lp::DoWhileStmt       (do-while)
└─ lp::RepeatStmt        (repeat-until)
└─ lp::ForStmt           (for-from-to-step)
└─ lp::SwitchStmt        (switch-case)
└─ lp::ExprNode          (abstracta para expresiones)
  └─ Constante           (NUMERO, CADENA, BOOLEANO)
  └─ VariableNode        (IDENTIFICADOR)
  └─ UnaryMinusNode
  └─ OrNode, AndNode, NotNode
  └─ EqualNode, NotEqualNode
  └─ LessNode, GreaterNode, LessEqualNode, GreaterEqualNode
  └─ AdditionNode, SubtractionNode
  └─ MultiplicationNode, DivisionNode
  └─ IntegerDivisionNode (//)
  └─ ModulusNode
  └─ PotenciaNode (^)
  └─ TernaryNode (?:)
  └─ FuncionNode        (sin, cos, sqrt, ...)

```

Métodos clave

```

class Nodo {
public:
    virtual void evaluar() = 0;
    virtual ~Nodo() {}
};

class ExprNode : public Nodo {
public:
    virtual double evaluateNumber(lp::Table& s) = 0;
    virtual bool evaluateBool(lp::Table& s) = 0;
    virtual std::string evaluateString(lp::Table& s) = 0;
    virtual Tipo getType(lp::Table& s) = 0;
    virtual void evaluar() override;
};

```

TernaryNode (al final del fichero)

```

class TernaryNode : public ExprNode {
public:
    ExprNode* condicion;

```

```

ExprNodo* verdadero;
ExprNodo* falso;
TernaryNode(ExprNodo* c, ExprNodo* v, ExprNodo* f)
    : condicion(c), verdadero(v), falso(f) {}
Tipo getType(lp::Table& s) override;
double evaluateNumber(lp::Table& s) override;
bool evaluateBool(lp::Table& s) override;
std::string evaluateString(lp::Table& s) override;
};

```

Colocación: TernaryNode se pone al final del fichero ast.hpp (justo antes de } del namespace) para que el script setup_b pueda borrarlo limpiamente con sed durante las pruebas.

5. Fichero 4: ast.cpp (AST — implementación)

Evaluación de expresiones aritméticas

Cada nodo llama recursivamente a evaluateNumber de sus hijos:

```

double AdditionNode::evaluateNumber(lp::Table& s) {
    return izq->evaluateNumber(s) + der->evaluateNumber(s);
}

```

IntegerDivisionNode::evaluateNumber :

```

double IntegerDivisionNode::evaluateNumber(lp::Table& s) {
    return floor(izq->evaluateNumber(s) / der->evaluateNumber(s));
}

```

Evaluación de expresiones booleanas

Similar, pero devuelve bool . getType() se usa para decidir qué método llamar:

```

Tipo ExprNodo::getType(lp::Table& s) {
    // Si es numérico → TIPO_NUMERO
    // Si es booleano → TIPO_BOOLEANO
    // Si es string → TIPO_CADENA
}

```

TernaryNode::evaluateNumber

```

double TernaryNode::evaluateNumber(lp::Table& s) {
    if (condicion->evaluateBool(s))
        return verdadero->evaluateNumber(s);
    else
        return falso->evaluateNumber(s);
}

```

RepeatStmt::evaluar

```
void RepeatStmt::evaluar() {
    do {
        cuerpo->evaluar();
    } while (!condicion->evaluateBool(s));
}
```

¡Ojo! La condición del `repeat-until` se evalúa **después** de ejecutar el cuerpo, y el bucle termina **cuando la condición es verdadera**.

ForStmt::evaluar

```
void ForStmt::evaluar() {
    double inicioValor = inicio->evaluateNumber(s);
    double finValor = fin->evaluateNumber(s);
    double pasoValor = paso ? paso->evaluateNumber(s) : 1.0;

    NumericVariable* var = static_cast<NumericVariable*>(variable);
    for (double i = inicioValor; i <= finValor; i += pasoValor) {
        var->setValue(i);
        cuerpo->evaluar();
    }
}
```

PreIncrementStmt::evaluar y PreDecrementStmt::evaluar

```
void PreIncrementStmt::evaluar() {
    NumericVariable* var = static_cast<NumericVariable*>(variable);
    var->setValue(var->getValue() + 1.0);
}

void PreDecrementStmt::evaluar() {
    NumericVariable* var = static_cast<NumericVariable*>(variable);
    var->setValue(var->getValue() - 1.0);
}
```

AssignIncrementStmt y AssignDecrementStmt

```
void AssignIncrementStmt::evaluar() {
    NumericVariable* var = static_cast<NumericVariable*>(variable);
    double nuevoValor = var->getValue() + expr->evaluateNumber(s);
    var->setValue(nuevoValor);
}
```

6. Fichero 5: `table/init.hpp` (tabla de símbolos)

Registro de palabras reservadas

```
const char* keyword[] = {
    "read",
    "read_string",
    "print",
    "if",
    "then",
    "else",
    "end_if",
    "while",
    "do",
    "end_while",
    "repeat",      // ← necesario para pregunta D
    "until",      // ← necesario para pregunta D
    "for",
    "from",
    "step",
    "to",
    "end_for",
    "switch",
    "case",
    "default",
    "end_switch",
    "clear_screen",
    "place",
    "sin",
    "cos",
    "sqrt",
    "log",
    "log10",
    "exp",
    "integer",
    "abs",
    "or",
    "and",
    "not",
    "true",
    "false",
    "pi",
    "e",
    "gamma",
    "phi",
    "deg",
    ""
};
```

Cada palabra en este array se registra automáticamente en la tabla de símbolos durante la inicialización.

Si añades un nuevo keyword (ej. `repeat` / `until`), tienes que añadirlo aquí y como `%token` en `interpreter.y` .

Cómo `lookupSymbol` distingue keywords de variables

```
Variable* Table::lookupSymbol(const char* s) {
    // Primero busca en la tabla
    for (int i = 0; i < numVariables; i++) {
        if (strcasecmp(vars[i]->getName(), s) == 0)
            return vars[i];
    }
    return NULL; // no encontrado → error en lexer
}
```

Importante: los keywords **no son variables**. El lexer devuelve directamente el token correspondiente (`TOK_PRINT` , `TOK_IF` , etc.) sin llamar a `lookupSymbol` . El array `keyword[]` solo se usa para crear los `Keyword` objects en la tabla.

7. Flujo de evaluación

1. `main.cpp` llama a `yyparse()` .
2. **Bison** construye el AST: cada regla sintáctica crea un `Nodo*` con `new` .
3. La raíz del AST se guarda en `programa` (variable global en `interpreter.y`).
4. `main.cpp` llama a `programa->evaluar()` (método virtual de `Nodo`).
5. `Programa::evaluar()` itera sobre sus hijos (las sentencias) y llama a `evaluar()` de cada una.
6. Cada sentencia (ej. `PrintStmt` , `AssignStmt` , `IfStmt`) llama a métodos de evaluación de sus sub-expresiones.
7. Las expresiones devuelven valores numéricos, booleanos o strings según `getType()` .

Print (ejemplo de dispatch)

```
PrintStmt::evaluar()
→ expr->getType(s)
→ si TIPO_NUMERO → expr->evaluateNumber(s) → imprime número
→ si TIPO_BOOLEANO → expr->evaluateBool(s) → imprime true/false
→ si TIPO_CADENA → expr->evaluateString(s) → imprime string
```

8. Trampas de examen

1. `$N` cambia al añadir `controlSymbol`

Si añades un `controlSymbol: /* empty */` en una regla como:

```
bloque_do_while: DO controlSymbol sentencia WHILE expresion
```

El `$3` original (sentencia) pasa a ser `$4` . **Actualiza siempre los `$N`** después de tocar reglas.

2. `%right '?' ':'` debe ir ANTES de los relacionales

No lo pongas entre los operadores relacionales (los `%left < > <= >=`). Debe ir **al principio**, justo después de la declaración:

```
%right '?' ':'  
%left OR  
%left AND  
...
```

3. El `;` es opcional, pero mejor ponerlo

La regla `línea: sentencia PUNTO_Y_COMA | sentencia` permite omitir `;`, pero si la gramática espera `;` en ciertos lugares y no lo pones, puede haber conflictos.

4. `init.hpp` es OBLIGATORIO para nuevos keywords

Si añades `repeat / until` (pregunta D):

- En `interpreter.y`: `%token REPEAT UNTIL`
- En `table/init.hpp`: añadir "repeat" y "until" al array `keyword[]`
- En `interpreter.l`: regla "repeat" { return REPEAT; }

Sin el paso de `init.hpp`, el lexer no reconoce la palabra → el examen falla. Fue lo que le costó al usuario 0.5/10.

5. `//` (división entera) → `DIVISION_ENTERA`

- En `.l`: `"//" { return DIVISION_ENTERA; }`
- En `.y`: `%token DIVISION_ENTERA`
- Precedencia: al mismo nivel que `'*' y '/'`:

```
%left '*' '/' DIVISION_ENTERA MOD
```

- En AST: `IntegerDivisionNode` (hereda de `ExprNode`)
- Implementación: `double IntegerDivisionNode::evaluateNumber(lp::Table& s) { return floor(izq->evaluateNumber(s) / der->evaluateNumber(s)); }`

6. Los operadores `++ / -- / += / -=` son unarios o de asignación, no expresiones

No los trates como operadores dentro de `expresion`. Son sentencias independientes:

```
sentencia_incdec: INCREMENTO IDENTIFICADOR  
                | DECREMENTO IDENTIFICADOR  
                | IDENTIFICADOR INCREMENTO_ASIGNACION expresion  
                | IDENTIFICADOR DECREMENTO_ASIGNACION expresion  
                ;
```

7. `do ... while` no tiene `end_while`

A diferencia del `while` normal que termina con `end_while`, el `do ... while` termina tras la expresión del `WHILE`:

```
bloque_do_while: DO sentencia WHILE expresion
```

8. Errores semánticos en la tabla de símbolos

Si una variable no está declarada, el tipo es `TIPO_ERROR`. Siempre comprueba que `getType()` no sea `TIPO_ERROR` antes de operar.

9. Casteo de `Variable*` a `NumericVariable*`

Para acceder a `getValue()` / `setValue()`, necesitas castear:

```
NumericVariable* var = static_cast<NumericVariable*>(variable);  
var->setValue(var->getValue() + 1.0);
```

Si el tipo no es numérico, esto lanza una excepción. Asegúrate de que el tipo sea correcto.

10. El `for` itera con `double`, no `int`

El `ForStmt` usa `double` para el contador. Si haces `for i from 1 to 10 step 1`, el último valor será `10.0`, y la comparación `<=` funciona. Pero ten cuidado con errores de precisión si el paso no es entero.

11. Añadir/eliminar constantes predefinidas (E1/E2)

Las constantes se definen en los arrays de `init.hpp`. No hay que tocar `.1`, `.y`, ni el `AST`:

- **Núméricas** → array `numericConstant[]` en `init.hpp` (token `CONSTANT`, tipo `NUMBER`)
- **Lógicas** → array `logicalConstant[]` en `init.hpp` (token `CONSTANT`, tipo `BOOL`)

```
// Anadir constante numérica (solo init.hpp):  
numericConstant[] = {  
    {"PI",      3.14159265358979323846},  
    {"TAU",     6.283185307179586},      // ← nueva  
    {"",        0}  
};  
  
// Anadir constante lógica (solo init.hpp):  
logicalConstant[] = {  
    {"true", true},  
    {"false", false},  
    {"yes", true},                        // ← nueva  
    {"",      0}  
};
```

El `init()` en `init.cpp` itera genéricamente sobre estos arrays — no se toca.

El lexer ya reconoce cualquier identificador y devuelve su token (`CONSTANT`).

`ConstantNode::evaluateNumber()` / `evaluateBool()` busca el símbolo en la tabla y llama a `getValue()`.

Trampa: al usar `sed` para modificar `init.hpp`, cuidado con `{"", 0}` que aparece en varios arrays. Usa un patrón único como `/logicalConstant\[ \] = {/a` para añadir tras la línea de apertura del array

correcto.

9. Checklist rápido

Para cualquier pregunta del examen:

- ☐ En `interpreter.1` : regla para el/los nuevos tokens (operador o palabra reservada)
- ☐ En `interpreter.1` : ¿hace falta un estado exclusivo? (ej. para strings)
- ☐ En `interpreter.y` : `%token` para cada nuevo token
- ☐ En `interpreter.y` : precedencia del nuevo operador (si aplica)
- ☐ En `interpreter.y` : regla(s) sintáctica(s) que usan el nuevo token
- ☐ En `table/init.hpp` : array `keyword[]` actualizado (solo si son palabras reservadas)
- ☐ En `table/init.hpp` : arrays `numericConstant[]` o `logicalConstant[]` (si son constantes)
- ☐ En `ast.hpp` : nueva clase AST (solo si el nodo no existe ya)
- ☐ En `ast.cpp` : implementación del nuevo nodo
- ☐ Verificar **0 sr conflicts** con `make 2>&1 | grep conflict`
- ☐ Probar con un ejemplo mínimo

Si modificas el AST:

- ☐ La nueva clase hereda de `ExprNodo` (expresiones) o `Nodo` (sentencias)
- ☐ Implementa TODOS los métodos virtuales: `evaluateNumber` , `evaluateBool` , `evaluateString` , `getType` , `evaluar`
- ☐ `getType` devuelve el tipo correcto (el ternario, por ejemplo, inspecciona sus hijos)
- ☐ Liberas memoria en el destructor si es necesario
- ☐ El constructor coincide con los argumentos que le pasas desde `.y`

Documento generado para el Simulacro de examen de PL

Última actualización: Junio 2026

Si encuentras errores, actualiza este fichero y regenera el PDF con:

```
npx md-to-pdf fase0_completa.md
```